Algorithmes sur les arbres binaires et sur les arbres binaires de recherche

Cours

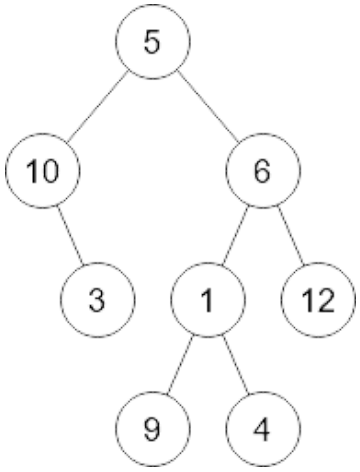
Un arbre est une structure de données **hiérarchique** (les nœuds sont liés par des relations père-fils) et récursive.

Un **arbre binaire** (**AB**) est un cas particulier d'arbre où chaque nœud possède au **maximum deux fils ordonnés** : un fils droit et/ou un fils gauche. Ils ne sont pas intervertibles !

On a déjà implémenté un arbre binaire et ses nœuds en Python par les classes AB et Noeud :

```
class AB:
    def __init__(self, racine = None):
        self.racine = racine # type : Noeud

class Noeud:
    def __init__(self, v, g = None, d = None):
        self.g = g # type Noeud
        self.d = d # type Noeud
        self.v = v
```



Implémentons l'arbre ci-contre.

```
n9, n4 = Noeud(9), Noeud(4)
n1, n12 = Noeud(1, n9, n4), Noeud(12)
n6 = Noeud(6, n1, n12)
n3 = Noeud(3)
n10 = Noeud(10, d = n3)
arbre = AB(Noeud(5, n10, n6))
```

Calcul de la taille d'un arbre binaire

Cours

La taille d'un arbre est son nombre de nœuds

Pour calculer la taille d'un arbre, on parcourt toutes les branches et compter le nombre de nœuds à partir de la racine.

Pour un arbre binaire, on peut créer une méthode récursive de la classe `Noeud` : `taille()`. Cette méthode renvoie la somme des tailles de ses deux branches. Pour un arbre binaire (classe `AB`) on ajoute une autre méthode appelée aussi `taille()` qui renvoie `0` pour l'arbre vide ou sinon la taille du Nœud racine.

```
class AB:
    def taille(self):
        if self.racine == None: return 0
 # renvoie la taille du nœud racine
        return self.racine.taille()

class Noeud:

    def taille(self):
        # taille du sous arbre droit
        td = 0 if self.droite is None else self.droite.taille()
        # taille du sous arbre gauche
        tg = 0 if self.gauche is None else self.gauche.taille()
        return 1 + td + tg
```

Calculons la taille de l'arbre précédent :

```
>>> arbre.taille()
8
```

Le calcul de la taille a un coût proportionnel à la taille du nombre de nœuds n de l'arbre (pour chaque nœud supplémentaire, on rajoute 3 opérations). La **complexité du calcul de la taille est en** .

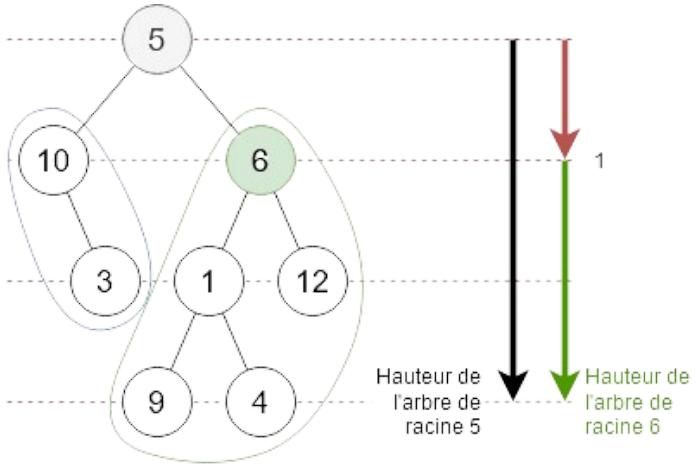
Calcul de la hauteur d'un arbre binaire

Cours

- La **profondeur** d'un nœud est le nombre d'arêtes de la racine au nœud. La profondeur de la racine est donc 0.
- La **hauteur** d'un arbre est la plus grande profondeur d'une feuille de l'arbre. Un arbre réduit à la racine a une hauteur de 0, un arbre vide a une hauteur de -1 (par convention).

Pour calculer la hauteur d'un arbre, il faut parcourir toutes ses branches et retenir la profondeur de la feuille la plus éloignée.

Pour un arbre binaire, on peut de la même façon que pour la taille, créer une méthode récursive de la classe `Noeud` : `hauteur()`. Cette méthode renvoie la plus grande des hauteurs de ses deux branches plus 1. Pour un arbre binaire (classe `AB`) on ajoute une autre méthode appelée aussi `hauteur()` qui renvoie `-1` pour l'arbre vide ou sinon la hauteur du Nœud racine.




```
class AB:

    def hauteur(self):
        if self.racine == None: return -1
        # renvoie la hauteur du nœud racine
        return self.racine.hauteur()

class Noeud:
    def hauteur(self):
        # hauteur du sous arbre droit
        td = -1 if self.droite is None else self.droite.hauteur()
        # hauteur du sous arbre gauche
        tg = -1 if self.gauche is None else self.gauche.hauteur()
        return 1 + max(td , tg)
```

Le calcul de la hauteur a un coût proportionnel à la taille du nombre de nœuds n de l'arbre (pour chaque nœud supplémentaire, on rajoute 3 opérations). La **complexité du calcul de la hauteur est en** .

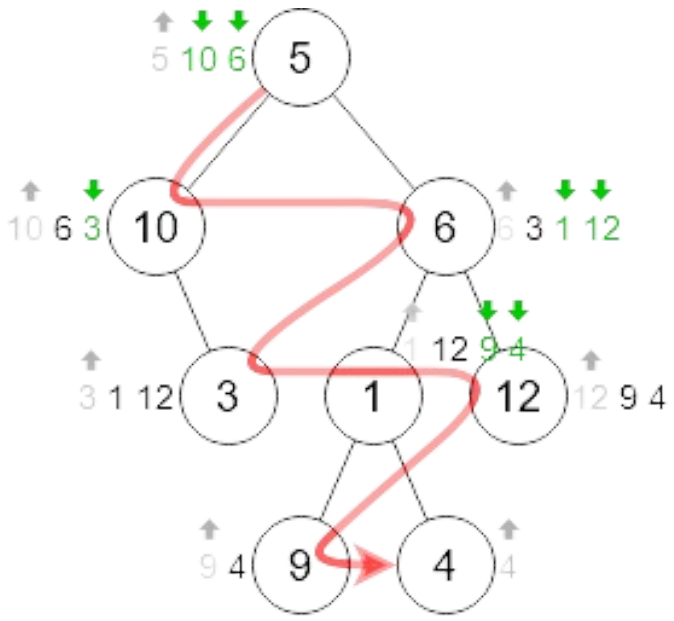
Parcourir un arbre binaire

Parcourir un arbre consiste à visiter chaque nœud en gardant en mémoire son fils gauche et son fils droit (dans cet ordre) avant de les parcourir à leur tour. Il existe de nombreux algorithmes de parcours qui visitent chaque nœuds dans un **ordre différent**, les plus courants[1] sont le parcours en largeur et le parcours en profondeur.

Parcours en largeur (BFS)

Cours

Le **parcours en largeur** ou **BFS** (*Breadth First Search*), consiste à parcourir l'arbre **niveau par niveau**. Les nœuds de niveau 0 sont d'abord parcourus puis les nœuds de niveau 1 et ainsi de suite. Dans chaque niveau, les nœuds sont parcourus de la **gauche vers la droite**.



L'implémentation va nécessiter l'utilisation une structure en file : pour chaque nœud visité on met en attente son fils de gauche puis son fils de droite dans une file (enfiler) puis on visite les nœuds présents dans la file (défiler).

La méthode est itérative, on peut donc l'ajouter dans la classe Nœud ou dans la classe AB directement:

```
class Noeud:
    def parcours_largeur(self):
        parcours = []
        file = []         # file de noeuds en attente
        file.append(self)
        while len(file) != 0:   # tant que la file n'est pas vide
            n = file.pop(0)             # on défile le premier nœud de la file
            parcours.append(n.valeur)   # on note sa valeur
            if n.gauche is not None: file.append(n.gauche)  #on enfile le fils gauche
            if n.droite is not None: file.append(n.droite)  #on enfile le fils droite
        return parcours
```

et l'appel de la méthode pour le nœud racine :

```
class AB :
    def parcours_largeur(self):
        if self.racine is None: print("l'arbre est vide")
        else: return self.racine.parcours_largeur()
```

Parcours en profondeur (DFS)

Cours

Le **parcours en profondeur** ou **DFS** (*Depth First Search*) consiste à parcourir pour chaque nœud d'abord le sous-arbre gauche entièrement et ensuite le sous arbre droit.

Chaque nœud est parcouru trois fois, on peut donc décider de traiter le nœud :

- Avant le parcours de son sous-arbre gauche : parcours **préfixe** (ou **préordre**) ;
- Entre le parcours de sous-arbre gauche et celui de son sous-arbre droit : parcours **infixe** (ou **en ordre**) ;
- Après le parcours de son sous-arbre droit : parcours **postfixe** (ou **suffixe**, ou encore **postordre**).

Parcours préfixe

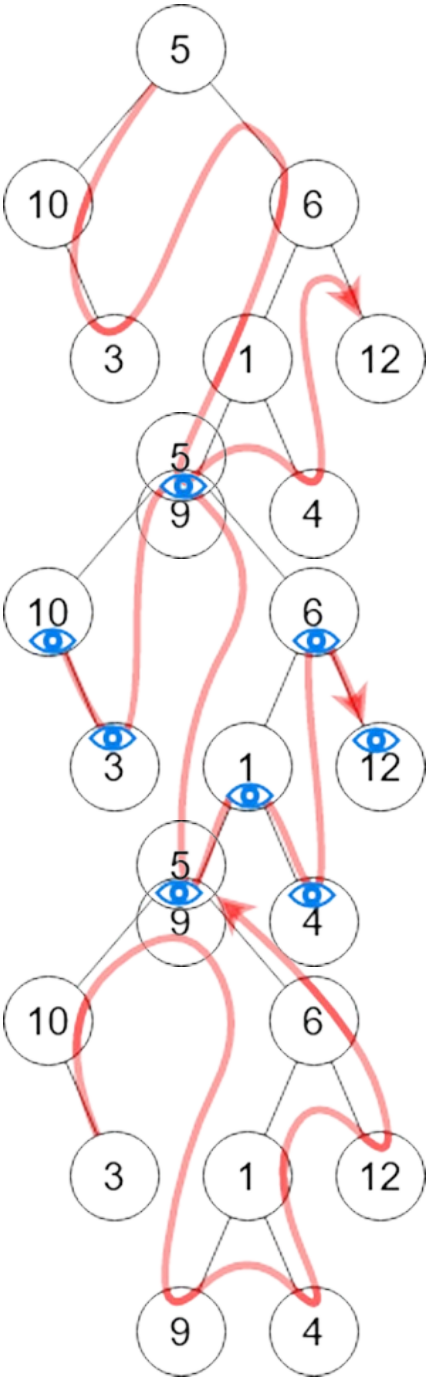
1. Visite du nœud
2. Parcours branche gauche
3. Parcours branche droite

Parcours infixe

1. Parcours branche gauche
2. Visite du nœud
3. Parcours branche droite

Parcours postfixe

1. Parcours branche gauche
2. Parcours branche droite
3. Visite du nœud



Les implémentations récursives de ces trois types de parcours en profondeur sont très semblables, seul change l'ordre des instructions. Observons la ligne `print(self.valeur, end='-')` :

Parcours préfixe

```
class AB :
    def parcours_prof(self):
        if self.racine is None: print("l'arbre est vide")
        else: return self.racine.parcours_prof()

class Noeud:
    def parcours_prof(self):

        print(self.valeur, end='-')       # on traite le noeud avant son sous-arbre gauche

        if self.gauche is not None:
            self.gauche.parcours_prof()

        if self.droite is not None:
            self.droite.parcours_prof()

>>> arbre.parcours_profondeur()
5-10-3-6-1-9-4-12-
```

Parcours infixe

```
class AB :
    def parcours_prof(self):
        if self.racine is None: print("l'arbre est vide")
        else: return self.racine.parcours_prof()

class Noeud:
    def parcours_prof(self):


        if self.gauche is not None:
            self.gauche.parcours_prof()

        print(self.valeur, end='-')       # on traite le noeud entre ses sous-arbre gauche et
droit

        if self.droite is not None:
            self.droite.parcours_prof()

>>> arbre.parcours_profondeur()
10-3-5-9-1-4-6-12-
```

Parcours postfixe

```
class AB :
    def parcours_prof(self):
        if self.racine is None: print("l'arbre est vide")
        else: return self.racine.parcours_prof()

class Noeud:
    def parcours_prof(self):


        if self.gauche is not None:
            self.gauche.parcours_prof()
```

```
        if self.droite is not None:
            self.droite.parcours_prof()

        print(self.valeur, end='-')       # on traite le noeud après son sous-arbre droit

>>> arbre.parcours_profondeur()
3-10-9-4-1-12-6-5-
```

On peut aussi implémenter le parcours préfixe de façon itérative, semblable au parcours en largeur à la différence qu'on utilise une pile plutôt qu'une file[2].

Algorithmes sur les arbres binaires de recherche

Cours

Un **arbre binaire de recherche** (**ABR**) est un cas particulier d'arbre binaire où :

- Chaque nœud a une **clé[3]: supérieure* à celles de tous les nœuds de son** sous-arbre gauche**
- Chaque nœud a une **clé inférieure** à celles de tous les nœuds de son **sous-arbre droit** Tous les **sous-arbres sont aussi des ABR**.

Note : « supérieur » et « inférieur » peuvent être au sens strict ou large en fonction de la définition donnée.

On considère l'ABR précédent qui servira comme support pour illustrer la suite :

Exemple d'ABR

A noter : Dans un ABR, le parcours infixe trie les valeurs des nœuds dans l'"ordre croissant.

La classe ABR est une sous classe d'AB qui hérite[4] des méthodes d'un arbre binaire et permet d'ajouter ses propres méthodes.

```
class ABR(AB):
    pass

a = ABR(Noeud(7, Noeud(4, Noeud(2), Noeud(6)), Noeud(11, Noeud(9, Noeud(8), Noeud(10)),
Noeud(12))))
```

Les méthodes de la classe AB fonctionnent par héritage :

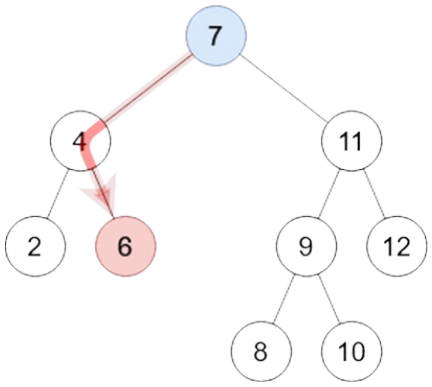
```
>>> a.taille()
9
>>> a.hauteur()
3
```

Rechercher une clé

Comme son nom l'indique, un ABR est spécifiquement adapté pour rechercher rapidement une clé. On parcourt l'ABR en partant de sa racine, et on observe trois cas :

- si la valeur recherchée est plus grande que la valeur du nœud parcouru, on descend à droite ;

- si la valeur recherchée est plus petite que la valeur du nœud parcouru, on descend à gauche ;
- si c'est la même, on renvoie le nœud.



Une fois arrivé à une feuille, la clé n'a pas été trouvée. L'exemple ci-contre montre la recherche de la valeur 6.

Modifions la classe `ABR`, pour gérer immédiatement le cas de l'arbre vide (la valeur n'est pas trouvée) et appeler une méthode de la classe `Noeud`.

```
class ABR(AB):
     def recherche(self, v):
        """ Renvoie True si v est une valeur de l'arbre"""
        if self.racine is None: return False
        return self.racine.recherche(v)
```

Voici ce que cela donne en récursif pour la classe `Noeud` :

```
class Noeud:
    def chercher(self, v):
        if self.valeur == v: return True       # la valeur est trouvée
        if self.valeur > v:      # on regarde à gauche
            if self.gauche is None: return False
            else: return self.gauche.chercher(v)
        if self.valeur < v:      # on regarde à droite
            if self.droite is None: return False
            else: return self.droite.chercher(v)
```

ou en itératif :

```
    def chercher(self, v):
        """ Renvoie le Noeud de valeur v
            None si aucun noeud n'a cette valeur
        """
        n = self.racine
        while n is not None:
            if v > n.valeur:
                n = n.droite
            elif v < n.valeur:
                n = n.gauche
            else:
                return n
```

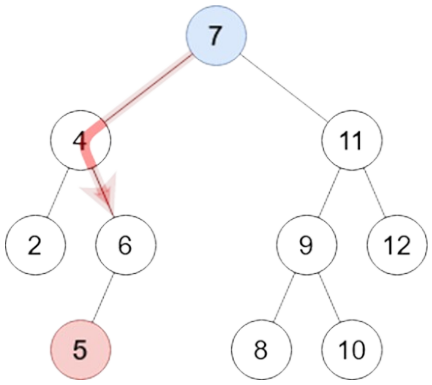
A chaque appel récursif, ou itération de la boucle `while`, le nombre de nœuds à parcourir est divisé par un facteur 2 (dans un arbre équilibré), on en déduit que la **complexité de la recherche d'une clé est en** .

Insérer une clé

Lorsqu'on ajoute un nœud à un ABR, on **ajoute toujours une feuille de l'arbre, jamais un nœud interne**.

Il suffit donc de rechercher la feuille dont la clé est la plus proche de celle du nœud à insérer (comme pour l'algorithme de recherche), puis d'ajouter le nœud à gauche si sa clé est inférieure à celle de la feuille, et à droite dans le cas contraire. L'exemple ci-contre montre l'insertion de la valeur 5.

Dans le cas d'un ABR qui n'autorise pas les clés multiples, il faut empêcher l'insertion d'un nœud dont la clé est déjà dans l'arbre.



Modifions la classe `ABR`, pour gérer immédiatement le cas de l'arbre vide (on ajoute une racine) et appeler une méthode de la classe `Noeud`.

```
class ABR(AB):
    def inserer(self, v):
        """ insere la valeur v dans l'arbre"""
        # si l'arbre est vide, on ajoute une racie
        if self.racine is None: self.racine = Noeud(v)
        else:
            self.racine.inserer(v)
```

Voici ce que cela donne en récursif pour la classe `Noeud` :

```
class Noeud:
    def inserer(self,v):
        """insere la valeur v dans l'ABR"""
        if v < self.valeur:
            if self.gauche is None: self.gauche = Noeud(v)    # on ajoute v à gauche
            else: self.gauche.inserer(v)    # on descend à gauche
        if v > self.valeur:
            if self.droite is None: self.droite = Noeud(v)    # on ajoute v à droite
            else: self.droite.inserer(v)    # on descend à droite
        if v == self.valeur: print(v,"est déjà dans l'ABR")   # la valeur est déjà dans l'ABR
```

ou en itératif :

```
    def inserer(self, v):
        """ Insère un nouveau Noeud de valeur v dans l'arbre    """
        if self.racine is None:
            self.racine = Noeud(v)
        else:
            n = self.racine
            while n is not None:
                if v < n.valeur:
                    if n.gauche is None:
                        n.gauche = Noeud(v)
                        return
                    n = n.gauche
                elif v > n.valeur:
                    if n.droite is None:
                        n.droite = Noeud(v)
                        return
                    n = n.droite
                else:
                    return # clé déjà dans l'arbre
```

La **complexité de l'insertion d'une clé est en** .

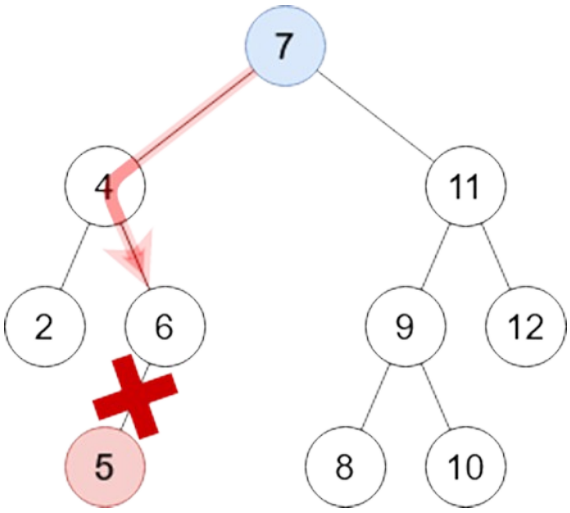
Supprimer une clé (hors programme)

L'opération dépend du nombre de fils du nœud à supprimer.

Cas n°1 : le nœud à supprimer est une feuille

- On cherche le père du nœud à supprimer ;
- On supprime le lien père-fils.

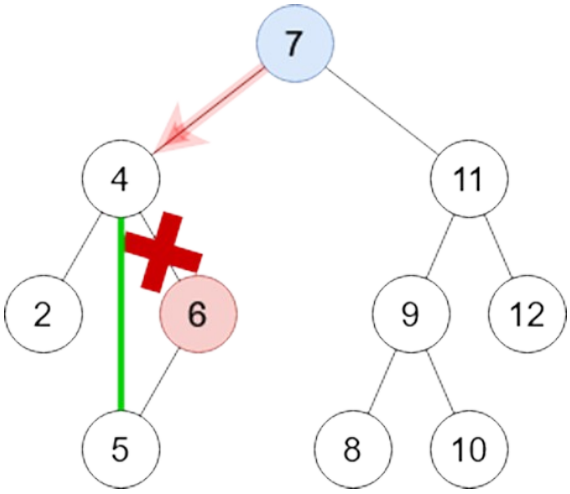
Exemple : suppression du nœud 5



Cas n°2 : le nœud à supprimer avec un seul fils

- On recherche le père du nœud à supprimer ;
- On redirige le lien père-fils vers le fils (unique) du nœud à supprimer.

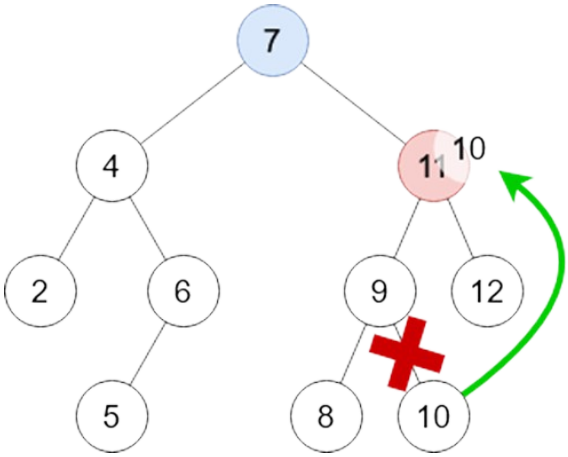
Exemple : suppression du nœud 6



Cas n°3 : le nœud à supprimer avec deux fils

- On recherche le nœud de sa branche de gauche ayant la plus grande valeur, comme c'est le plus grand de cette branche, il n'a pas de fils droit (voir valeur max) ;
- On rompt le lien père-fils, (comme dans le cas n°2 ;)
- Et on remplace la valeur du nœud à supprimer par celle du nœud décroché.

Exemple : suppression du nœud 11



Remarque : alternativement, on peut décrocher le nœud ayant la plus petite valeur de sa branche de droite.

```
def supprimer(self, v):
    """ Supprime le Noeud de valeur v dans l'arbre
        (s'il existe)
    """
    # on recherche le noeud à supprimer n et son pere p
    p= None
    n = self.racine
    while n is not None:
        if v > n.valeur:
            p = n
            n = n.droite
        elif v < n.valeur:
            p = n
            n = n.gauche
        else:
            break
    if n is None: return -1  # la valeur n'existe pas
    fils = 0
    if n.gauche is not None: fils += 1
    if n.droite is not None: fils += 1

    print(fils)
    print(p.valeur)

    if n.gauche is None and n.droite is None:    # n est une feuille
        if p.droite == n: p.droite = None
        else: p.gauche = None
    elif n.gauche is not None and n.droite is None:  # 1 seul fils à gauche
        if p.droite == n: p.droite = n.gauche
        else: p.gauche = n.gauche
    elif n.gauche is  None and n.droite is not None:  # 1 seul fils à droite
        if p.droite == n: p.droite = n.droite
        else: p.gauche = n.droite
    else:                # 2 fils
        # on cherche le max et son père
        max = n.gauche
        p = n
        while max.droite is not None:
            p = max
            max = max.droite
        p.droite = max.gauche     # on rattache la gauche du max à p
        n.valeur = max.valeur     # on remplace la valeur de n par le max
```

1. D'autres algorithmes existent, par exemple la recherche arborescente Monte-Carlo. ↩

2.
```
class Noeud:
def parcours_prefixe(self):
    parcours = []
     pile = []
    pile.append(self)
    while len(pile) != 0:   # tant que la pile n'est pas vide
        n = pile.pop()   # on prend le noeud au sommet de la pile
        parcours.append(n.valeur)    # on note sa valeur
        if n.droite is not None: pile.append(n.droite)  #on empile le fils DROIT d'abord
        if n.gauche is not None: pile.append(n.gauche)  #on empile le fils gauche
```

   ↩

3. Dans un ABR, l'étiquette est appelée « clé » ou « valeur ». ↩

4. L'héritage est un des grands principes de la programmation orientée objet (POO) permettant de créer une nouvelle classe à partir d'une classe existante. La sous classe hérite des attributs et des méthodes de la classe mère et en ajoute de nouveaux. ↩